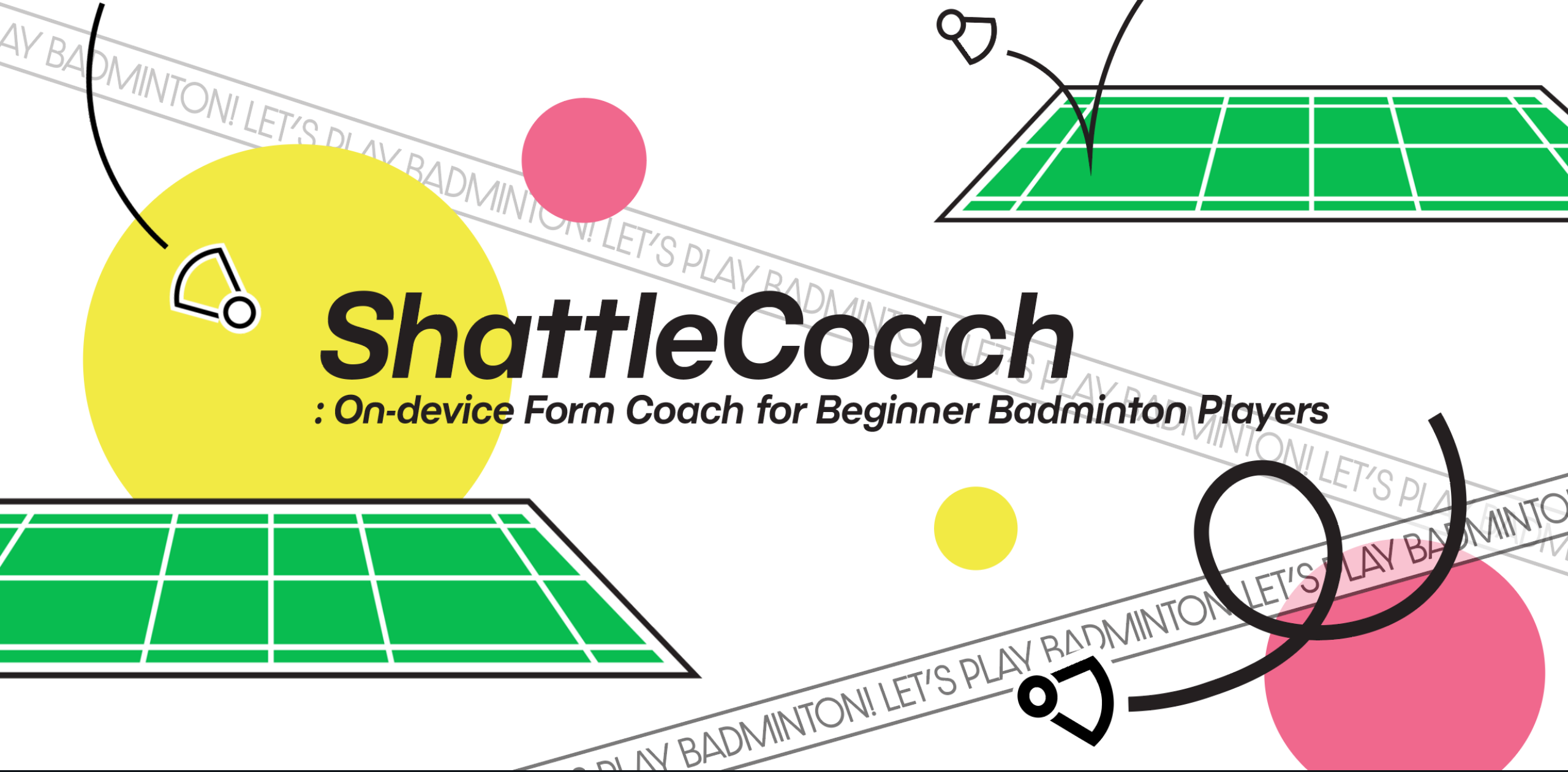




ShuttleCoach

: On-device Form Coach for Beginner Badminton Players

Coaching that doesn't scale — solved on the phone



The problem

University clubs absorb dozens of beginners each semester, but a few seniors can't watch every swing. Bad habits — frypan grip, dropped elbow, late impact — set in within a month.



Our goal

From a 5-second swing clip, return per-criterion form feedback for three foundational strokes — **all inference on-device, no video leaves the phone.**

Built for real use at Shattlecock (SNU CSE badminton club): form coaching + everything club operations needs.

Usage scenario

1

Mount & pick

Tripod ~3 m from the baseline, open the app, choose a stroke.

2

Record 5 s

Live pose-skeleton overlay confirms a good side-on framing.

3

On-device analysis

MoveNet + scorer run locally (~24 s on Galaxy S21).

4

Read the scorecard

Stars, impact-frame annotation, and a short Korean coaching note.

A three-stage on-device pipeline



1 · Pose extraction

MoveNet Lightning (TFLite INT8, ~2.9 MB)
→ 17 COCO keypoints per frame, drawn live
on the preview.



2 · Form scoring

Rule-based geometry on 4 criteria (impact
point · elbow · waist rotation · knee) + BST
percentile lookup for speed & footwork. Axes
adapt per stroke.



3 · Visualization

Per-axis 1–5★, an annotated impact frame,
“what you did well,” and a deterministic
Korean coaching note (optional LLM polish).

Three strokes, each with its own scoring axes

High clear

Short serve

Forehand drive

Stroke-conditional axes (e.g. footwork is dropped for short serve) keep each score meaningful.

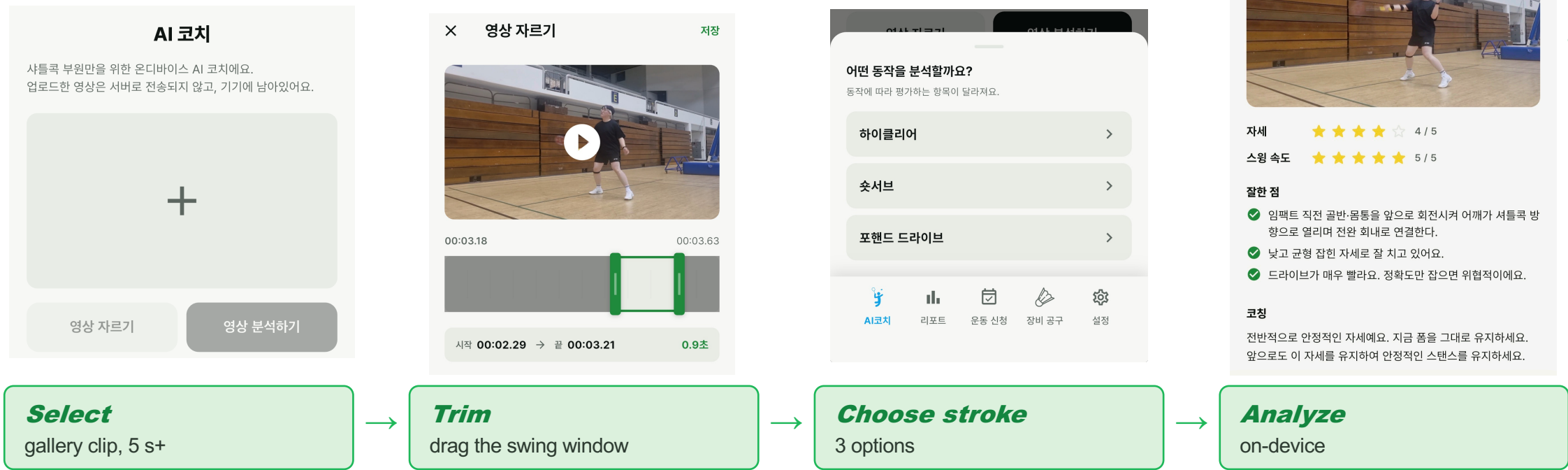


Why pose input?

5 s of HD video → ~5 KB of keypoints. Privacy by design
(nothing stored or sent) and small enough for real-time on-
device inference.

AI Coach — the core loop

03



The scorecard

★ Per-axis star ratings

Posture (4-criteria geometry), swing speed & footwork (BST percentiles).

✓ What you did well

1–3 short positive lines with check icons — encouragement first.

🏃 Impact-frame annotation

The impact moment with circles, arrows and a Korean cue overlaid.

🔧 Korean coaching note

Deterministic rulebook sentence; optional Groq llama-3.1-8b polish, guardrailed (no new advice).

DEMO

See it in action



YouTube Short demo

youtube.com/shorts/_lbC8qLcBfQ

The full user journey

1

Onboarding

Google login → intro → profile (name, handedness)

2

AI Coach

Pick clip → trim → choose stroke → on-device scorecard

3

Reports

Monthly heatmap of progress; tap a day for records

4

Sessions

Browse the week, RSVP / cancel practice slots

5

Equipment

Group-buys & member sharing board

Captured on a iPhone 15 · scan or open the link for the 30-second walkthrough.

07



Not just a coach — a club platform

ShuttleCoach already runs Shattlecok's day-to-day. Five tabs, one app, Google login over Supabase.



AI Coach

Swing video → on-device form scorecard.



Reports

Monthly heatmap calendar of practice & progress; tap a day for records.



Sessions

Weekly schedule with RSVP / cancel, capacity, attendee list (Supabase).



Equipment

Community board: official group-buys (exec-only) and member sharing.



Settings

Profile, executive-code registration, OAuth, licenses.



Auth & roles

Google OAuth + guest (grader) login; row-level security & exec roles.

Beyond the assignment: a real product we're polishing to actually manage the club next semester.

Surviving the real world



Our ST-GCN stroke classifier scored 89.6% on broadcast data but collapsed to 28.6% on real phone clips — a domain gap (the model leaned on a confidence=1 crutch absent in noisy MoveNet keypoints).

28.6%



82.9%

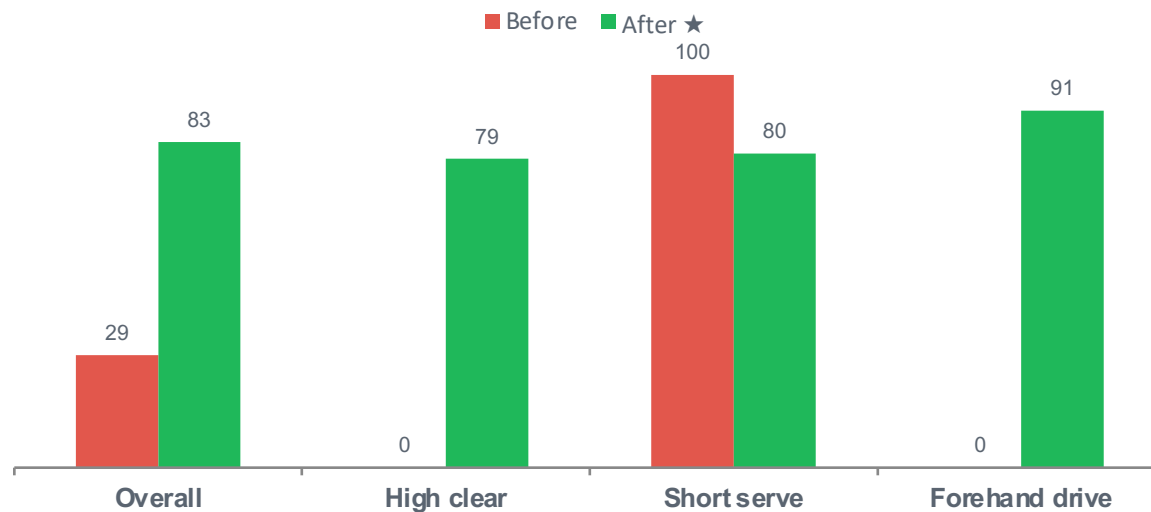
before (real phone)

after retraining

Honest trade-off: broadcast accuracy fell 89.6% → 49.4% — the model became a “phone expert,” which is all the app sees (test n=35, small).

How: domain-adaptation retraining

- Randomize keypoint confidence + random joint-drop → kill the conf=1 crutch
- Augment to mimic MoveNet noise (flip · time-warp · jitter · drop)
- Oversample real phone clips; boost the weak class (1.8× loss)



Optimizing for the phone: size & speed

 **Quantization — the real work was making INT8 run on mobile**

INT8 scheme	Size	Accuracy	On mobile onnxruntime
Naive INT8 (poor calibration)	327 KB	-20.89 pp	rejected — too costly
Dynamic INT8 (weight-only)	308 KB	89.7% (desktop)	ConvInteger unsupported → crash
QDQ static INT8 ★	327 KB	-2.9 pp (80.0%)	QuantizeLinear → runs ✓

3.2× smaller (1,061 → 327 KB) at only -2.9 pp once calibration was fixed and Conv wrapped in Q/DQ instead of the unsupported ConvInteger.

↔ **Latency — 3-isolate parallel inference**

51 s → 24 s

End-to-end on Galaxy S21, 1 → 3 Dart isolates (2.1× faster) — what makes on-device analysis practical.

 **Production decision:** the shipped scorer is rule-based (geometry + BST percentiles) for out-of-distribution robustness; the optimized neural classifier is retained for a “stroke-confirmation” feature and future release. Optimization scoring focuses on size-reduction rate and the marginal -2.9 pp drop.

Five weeks, two people

Project timeline (May 5 – Jun 1)

- W1** Rubric & repo; pilot labeling; Flutter scaffold + pose overlay
- W2** Filming session 1; FP32 training; recording flow + scorecard shell
- W3** Filming session 2; baseline CV; on-device inference wired
- W4** Retraining & quantization study; latency / size benchmarks
- W5** Final report & figures; UI polish; APK build; demo video



Contribution



Both — self-collected, double-labeled swing dataset (filming + annotation).

Eunwoo Choi — App & integration & UI Design

Entire Flutter app: UI, routing, auth (Google/Supabase), Reports / Sessions / Equipment / Settings, native frame-extractor channels, wiring the ML pipeline into the app end-to-end.

Donghwi Moon — On-device ML

Model training & evaluation, domain-adaptation retraining, quantization & ONNX experiments, and the scoring + coaching logic.